

Projektdokumentation

Steam Library REST API

Modul M295 – Applikationen entwerfen und implementieren

Philipp Seewer

Inhaltsverzeichnis

1. Projektidee
2. Anforderungskatalog
3. Datenmodell & Klassendiagramm
4. REST-Schnittstellen
5. Storyboard & Screen Mockups
6. Testplan
7. Testdurchführung
8. Installationsanleitung
9. Hilfestellungen

1. Projektidee

Die Steam Library REST API ist eine serverseitige Anwendung zur Verwaltung einer persönlichen Spielebibliothek. Ziel des Projektes ist es, eine REST-basierte Schnittstelle bereitzustellen, mit der Spiele gespeichert, bearbeitet, gefiltert und gelöscht werden können.

Die Anwendung ermöglicht es, Informationen wie Titel, Steam-App-ID, Spielzeit, Installationsstatus sowie das zuletzt gespielte Datum zu verwalten.

Die REST-API speichert sämtliche Daten persistent in einer relationalen MySQL-Datenbank.

Das Projekt demonstriert die vollständige Umsetzung eines CRUD-Prozesses unter Verwendung von Spring Boot, JPA, Validierung und strukturierter Fehlerbehandlung.

2. Anforderungskatalog

Kernfunktionalität

Das System muss folgende Funktionen unterstützen:

- Spiel erfassen
- Spiel bearbeiten
- Spiel löschen
- Einzelnes Spiel anzeigen
- Liste aller Spiele anzeigen
- Spiele nach Titel filtern
- Spiele nach Installationsstatus filtern
- Spiele nach Mindestspielzeit filtern
- Spiele nach Datum filtern

User Stories

User Story 1

Als Benutzer möchte ich ein neues Spiel erfassen können, damit ich meine Bibliothek verwalten kann.

Akzeptanzkriterien:

- POST /games speichert ein Spiel in der Datenbank
- Ungültige Daten werden validiert
- Bei Fehlern wird eine verständliche Fehlermeldung zurückgegeben

User Story 2

Als Benutzer möchte ich meine Spiele bearbeiten können.

Akzeptanzkriterien:

- PUT /games/{id} aktualisiert bestehende Datensätze
- Nicht existierende IDs liefern 404
- Validierungsregeln werden überprüft

User Story 3

Als Benutzer möchte ich Spiele suchen und filtern können.

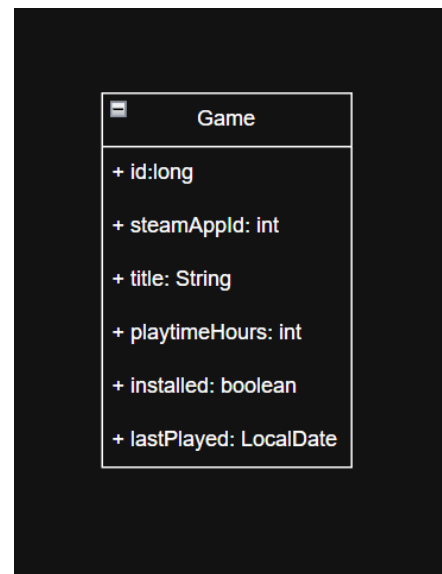
Akzeptanzkriterien:

- GET /games?title=
- GET /games?installed=
- GET /games?minPlaytime=
- GET /games?lastPlayedAfter=

3. Datenmodell & Klassendiagramm

Entität: Game

Attribut	Typ	Beschreibung
id	Long	Primärschlüssel
steamAppId	Long	Steam App ID
title	String	Titel des Spiels
playtimeHours	Integer	Spielzeit
installed	Boolean	Installiert oder nicht
lastPlayed	LocalDate	Letztes Spieldatum



Beziehung:

Es existiert aktuell eine einzelne Entität ohne weitere Relationen.

4. REST-Schnittstellen

GET /games

Lädt alle Spiele.

Optional:

- ?title=
- ?installed=
- ?minPlaytime=
- ?lastPlayedAfter=

GET /games/{id}

Lädt ein einzelnes Spiel anhand der ID.

POST /games

Erstellt ein neues Spiel.

Beispiel Body:

```
{  
  "steamAppId": 730,  
  "title": "Counter-Strike 2",  
  "playtimeHours": 120,  
  "installed": true,  
  "lastPlayed": "2026-02-19"  
}
```

PUT /games/{id}

Aktualisiert ein bestehendes Spiel.

DELETE /games/{id}

Löscht ein Spiel.

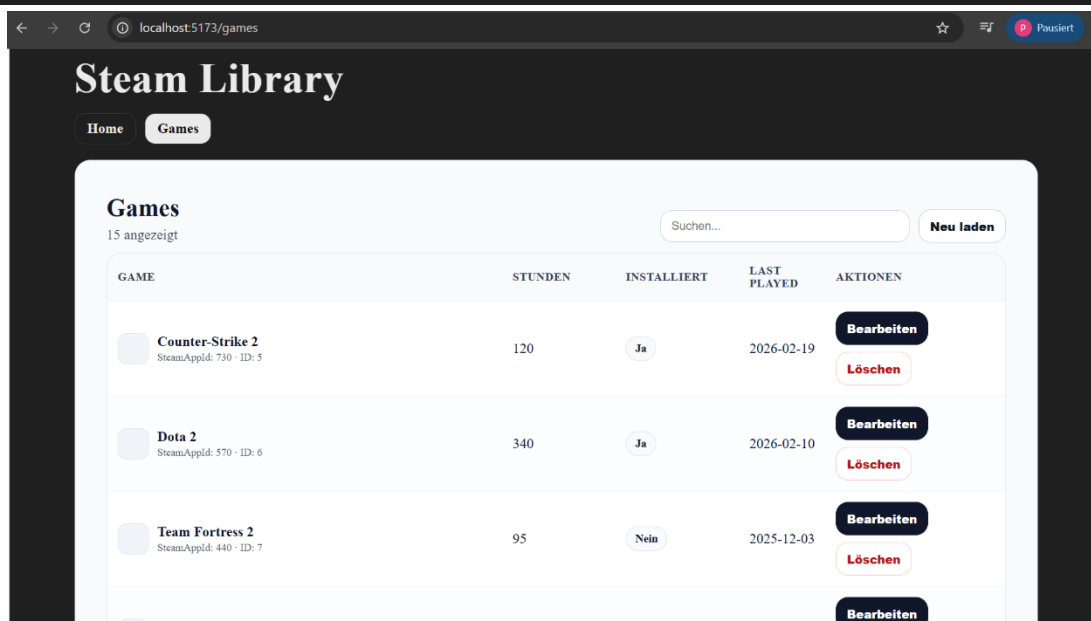
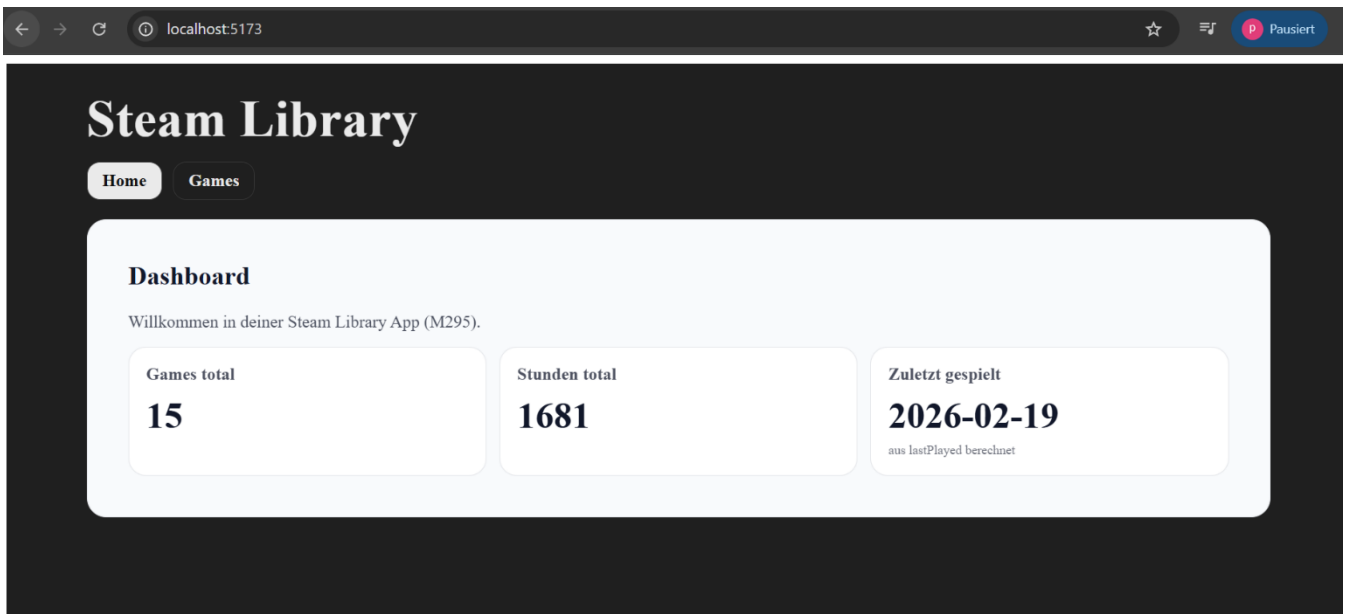
Fehlerbehandlung

- 400 – Validierungsfehler
- 404 – Ressource nicht gefunden
- 500 – Interner Serverfehler

Fehlermeldungen werden strukturiert als JSON zurückgegeben.

5. Storyboard & Screen Mockups

```
← → ↺ ⓘ localhost:8080/games
Quelltextformatierung ☐
[{"id":5,"title":"Counter-Strike 2","steamAppId":730,"playtimeHours":120,"installed":true,"lastPlayed":"2026-02-19"},{"id":6,"title":"Dota 2","steamAppId":570,"playtimeHours":340,"installed":true,"lastPlayed":"2026-02-10"},{"id":7,"title":"Team Fortress 2","steamAppId":440,"playtimeHours":95,"installed":false,"lastPlayed":"2025-12-03"},{"id":8,"title":"Grand Theft Auto V","steamAppId":271590,"playtimeHours":210,"installed":true,"lastPlayed":"2026-01-28"},{"id":9,"title":"Baldur's Gate 3","steamAppId":1086940,"playtimeHours":180,"installed":true,"lastPlayed":"2026-02-01"},{"id":10,"title":"Red Dead Redemption 2","steamAppId":1174180,"playtimeHours":75,"installed":false,"lastPlayed":"2025-11-18"},{"id":11,"title":"Cyberpunk 2077","steamAppId":1091500,"playtimeHours":120,"installed":true,"lastPlayed":"2026-02-06"},{"id":12,"title":"Valheim","steamAppId":892970,"playtimeHours":60,"installed":false,"lastPlayed":"2025-09-20"},{"id":13,"title":"Stardew Valley","steamAppId":413150,"playtimeHours":140,"installed":false,"lastPlayed":"2025-10-14"},{"id":14,"title":"ELDEN RING","steamAppId":1245620,"playtimeHours":98,"installed":true,"lastPlayed":"2026-01-05"},{"id":15,"title":"Dead by Daylight","steamAppId":381210,"playtimeHours":45,"installed":false,"lastPlayed":"2025-08-30"},{"id":16,"title":"God of War","steamAppId":1593500,"playtimeHours":52,"installed":false,"lastPlayed":"2025-12-22"},{"id":17,"title":"Portal","steamAppId":400,"playtimeHours":12,"installed":false,"lastPlayed":"2025-06-01"},{"id":18,"title":"Portal 2","steamAppId":620,"playtimeHours":24,"installed":false,"lastPlayed":"2025-06-15"},{"id":19,"title":"Deep Rock Galactic","steamAppId":548430,"playtimeHours":110,"installed":true,"lastPlayed":"2026-02-12"}]
```



6. Testplan

Nr Testfall	Erwartetes Resultat
1 POST gültiges Spiel	201 Created
2 POST ohne Titel	400 Validation Error
3 GET mit Filter	Gefilterte Liste
4 PUT bestehende ID	200 OK
5 DELETE bestehende ID	204 No Content

7. Testdurchführung

Alle Testfälle wurden manuell mit Postman getestet.

Ergebnis:

- Alle CRUD-Operationen funktionieren korrekt.
- Validierung verhindert ungültige Eingaben.
- Fehler werden sauber behandelt.
- Daten werden persistent gespeichert.

8. Installationsanleitung

8.1 Voraussetzungen

Für den Betrieb der Applikation werden folgende Komponenten benötigt:

- **Java 17**
- **Maven (Version 3.9 oder höher)**
- **MySQL (Version 8 oder höher)**
- (Optional für das Frontend) **Node.js 18 oder höher**

8.2 Datenbank einrichten

1. MySQL starten.
2. Neue Datenbank erstellen:

```
CREATE DATABASE steam_library;
```

Benutzer erstellen und Rechte vergeben:

```
CREATE USER 'steam_user'@'localhost' IDENTIFIED BY 'steam_pass';
```

```
GRANT ALL PRIVILEGES ON steam_library.* TO 'steam_user'@'localhost';
```

```
FLUSH PRIVILEGES;
```

Hinweis

Die Standard-Zugangsdaten in der Datei

Backend/src/main/resources/application.properties lauten:

```
spring.datasource.username=steam_user
```

```
spring.datasource.password=steam_pass
```

8.3 Backend starten

1. In den Ordner Backend wechseln (dort befindet sich die pom.xml).
2. Folgenden Befehl ausführen:

```
mvn clean install
```

```
mvn spring-boot:run
```

3. Die REST-API ist anschliessend erreichbar unter:

<http://localhost:8080>

8.4 Projekt in einer IDE öffnen

1. Projekt als **Maven-Projekt** importieren.
2. Abhängigkeiten automatisch laden lassen.
3. Die Hauptklasse SteamLibraryApplication starten.

8.5 Frontend starten (optional)

Das Frontend ist nicht Bestandteil der Bewertung, dient jedoch zur Visualisierung der REST-API.

Voraussetzungen

- Node.js 18+
- npm

Installation

Im Ordner Front-End:

```
npm install
```

Starten:

```
npm run dev
```

Das Frontend ist anschliessend erreichbar unter:

<http://localhost:5173>

Wichtig Das Backend muss vor dem Start des Frontends laufen:

9. Hilfestellungen

Folgende Hilfsmittel wurden verwendet:

- Unterrichtsmaterial M295
- Offizielle Spring Boot Dokumentation
- Maven Dokumentation
- MySQL Dokumentation
- Internetrecherche zur Fehlerbehandlung
- Austausch mit Mitlernenden
- Bug Fixing mit GPT